UA

UNIFIED ACCESSIBILITY • HIRING ASSIGNMENT

Full-Stack Intern Take-Home Assignment

Accessibility Issue Tracker

Item	Details
Effort	About 5 hours of focused work
Deadline	2 calendar days from receiving this brief
Tools	Part 1: a proper architecture-diagramming tool. Part 2: any free tools of your choice, running locally.
Submit	A Git repository with README.md, ARCHITECTURE.md (with your diagrams), ACCESSIBILITY.md, and tests

What you are building

Build a small website where an accessibility audit team can log, track, and resolve accessibility problems found on web pages. Think of a tiny audit tracker: a user records issues found on a web page, moves each issue through a workflow, and can see a log of status changes.

There are two parts. First you design a larger production version and draw it. Then you build a small version that runs on your own computer.

Expected effort is about 5 hours of focused work, and you have 2 calendar days to submit. Architecture and accessibility together make up about half of the score, so give both real attention even though the app is small. We value a clear, correct, accessible solution more than a long list of features. If something is unfinished, tell us what is missing.

Part 1: Architecture design (written, with diagrams)

Write a short ARCHITECTURE.md (or PDF), about two pages plus diagrams. Imagine your tracker is used in production by 500 teams (about 20,000 users) with up to 1 million issues, and that hundreds of issues can be created within a minute when an audit finishes. Explain how you would design that production system. You only build the small version in Part 2, but your design must be consistent with it.

You must draw the diagrams yourself, using the best tool you can find for architecture diagrams, one that supports labelled components and arrows and exports to an image (for example draw.io, Excalidraw, Lucidchart, Figma, or Mermaid). Hand-drawn sketches are not accepted. Include the exported images in ARCHITECTURE.md and name the tool you used. Include:

1. **Architecture diagram (required, drawn by you).** Show the main components of the production system, for example browser, web or CDN layer, backend API, database, authentication, and any cache, queue, or file storage you would add. Label every arrow with what flows along it, mark which parts you build locally in Part 2 and which are production-only, and add only components you can justify.

2. **Request flows.** Using your diagram, walk through two flows step by step: a user creating an issue, and a user changing an issue's status. Cover validation, authorization, the database write, and what the user sees on success and on failure.
3. **Data model (ER diagram).** Draw an entity-relationship diagram of your tables with their main fields, relationships, and constraints, and list at least three indexes you would add, with a reason for each. Show how you record status changes.
4. **API.** List your endpoints: method, path, what it does, who may call it, and the error status codes it can return. Explain where and how you enforce that users can only access their own data.
5. **Status workflow.** Show your statuses (at least three) and the allowed transitions as a small state diagram or table, and name one change you would block and why. Explain what happens if two users change the same issue's status at the same time.
6. **Scale and reliability.** At 1 million issues and a burst of hundreds of new issues in a minute, what would become slow first, and what would you do about it (for example pagination, indexes, caching)? What happens if the database is briefly unavailable?
7. **A new requirement.** The team now wants screenshot attachments on issues and an email to a team lead when a Critical issue is created. Which components would you add to your diagram, and how would the email be sent without slowing down the request?
8. **Accessibility in the design.** Say where in your architecture error messages and status announcements come from (for example the API's error format) and how the frontend turns them into text that a screen reader announces.
9. **Technology choices.** Name your backend, frontend, and database, and one alternative you considered for one of them, with the tradeoff. Say what you use locally in Part 2 and what you would use in production, and why they might differ.

Part 2: Working application

Build a smaller, working version of the tracker on your computer. These features are required:

- **Demo users.** Provide two seeded demo users and a simple way to sign in as either. No registration, email, or real authentication service is needed.
- **Issues.** A signed-in user can create, view, and edit their own issues. An issue has a title (required), a description, a page URL (required, valid http or https), a WCAG criterion (for example "1.1.1 Non-text content", chosen from a list or typed), a severity (Minor, Moderate, Serious, or Critical), a status, and created and updated times.
- **Filtering.** The issue list can be filtered by status and by severity.
- **Status workflow.** A user can change an issue's status following the workflow you described in Part 1. The backend must reject a change your workflow does not allow, with a clear message.
- **Status log.** Every status change is recorded (who, when, from which status to which) and shown on the issue's page.
- **Validation.** Inputs are validated in the backend, not only in the browser, and errors are returned in a useful form.
- **Ownership.** A user cannot see, edit, or change the status of another user's issues, even by typing a URL or calling the API directly.
- **Persistence.** Data is saved in a database (SQLite is fine) and is still there after you restart the app.

For Part 2 you may use any free tools of your choice: language, framework, database, libraries, and editor. Keep the setup simple enough that a reviewer can start it by following your README. A list page, a create and edit form, and an issue detail page are enough.

Accessibility requirements

Accessibility is a big part of how we judge this assignment. Aim for the relevant basics of WCAG 2.2 AA. We will try your app without a mouse, not just look at an automated score.

- Every button, link, form, and filter works with the keyboard alone. Focus is always visible, and tab order follows the visual order.
- Every input has a visible label. Required fields are marked with text, not only a symbol or color.
- Errors are explained in text next to the field, and a person using a screen reader can find them (for example by moving focus to an error summary or to the first invalid field).
- Messages such as "Issue saved" or "Could not save" are announced to screen readers without moving focus.
- Severity and status are shown as words, not only as colored badges.
- Each page has one main heading, headings are in a sensible order, and the page title changes from page to page. Include a "skip to main content" link.
- Use native HTML controls (button, a, select, input, table) before building custom ones. Present the issue list as a real table or list with proper headers.
- Text has a contrast ratio of at least 4.5:1, and interface components such as input borders and focus outlines at least 3:1.
- The layout works at 320px wide and at 200% zoom with no content cut off and no sideways scrolling.

Check your own work. Do a keyboard-only pass through every flow. Run one free automated checker (for example axe DevTools, Lighthouse, or WAVE). If you can, try a screen reader (NVDA on Windows, VoiceOver on Mac). Record what you did and what you found in ACCESSIBILITY.md. It is fine to have issues left; tell us honestly what they are.

What to submit

Submit within 2 calendar days:

- A Git repository with the working source code. Private is fine; give the reviewers access.
- **README.md:** how to set up and start the app, the demo accounts, how to run the tests, and the known limitations.
- **ARCHITECTURE.md** (or PDF): your Part 1 design, including your diagrams.
- **ACCESSIBILITY.md:** the keyboard checks you did, any automated or screen reader checks, and the issues still remaining.
- **Tests:** at least two meaningful ones: an invalid status change is rejected, and one user is blocked from another user's data.
- **AI note:** a short note (in the README or a separate file) saying whether and how you used AI tools.

Ground rules

- Everything runs locally with free tools. Do not host or deploy the app, use paid APIs, or use real personal data.
- You may use AI for ideas, design, debugging, and code. AI makes mistakes, and you are responsible for checking what you submit.
- You should understand every important part of your work. In the review call we may ask why you chose something, what its tradeoffs are, or ask you to make a small change live.

- If something is unclear, make a reasonable assumption and write it in the README.
- A small, reliable, accessible app and a thoughtful design are better than a large, incomplete system. If you run out of time, say what is unfinished.